Moveit Collision Detection

Moveit Collision Detection
- 1. Usage Environment
- 2. Driving the Real Machine
  - 2.1 Starting the Real Device
  - 2.2 Starting MoveIt2
- 3. Collision Detection
  - 3.1. Launch Command
  - 3.2. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine involves subscribing to the Moveit2 `/joint_states` topic to convert the joint state information of the robotic arm into control data for a real robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

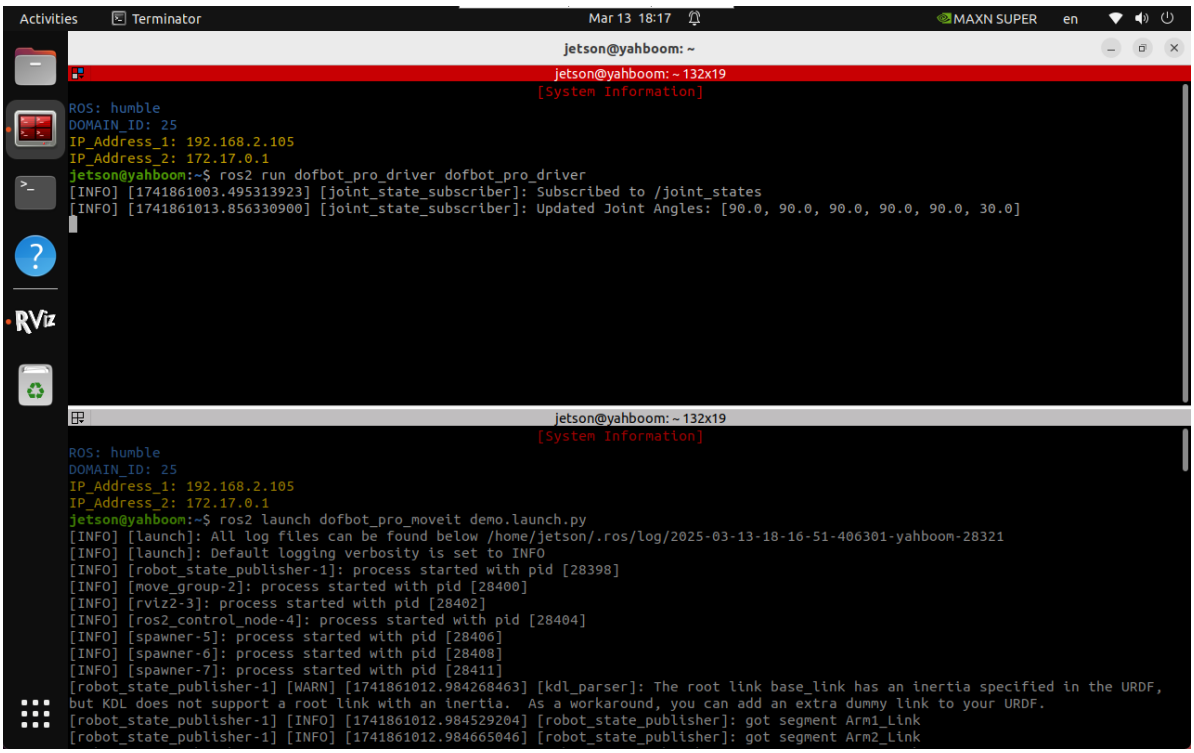
The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in the Docker container. (Both devices are pre-configured with the same `ros_domain_id`; simply ensure the virtual machine and the robotic arm are on the same network.)

2.1 Starting the Real Device

```
#Starting from Docker
ros2 run dofbot_ctrl SimulateToArm
```

2.2 Starting MoveIt2

```
ros2 launch dofbot_moveit demo.launch.py
```



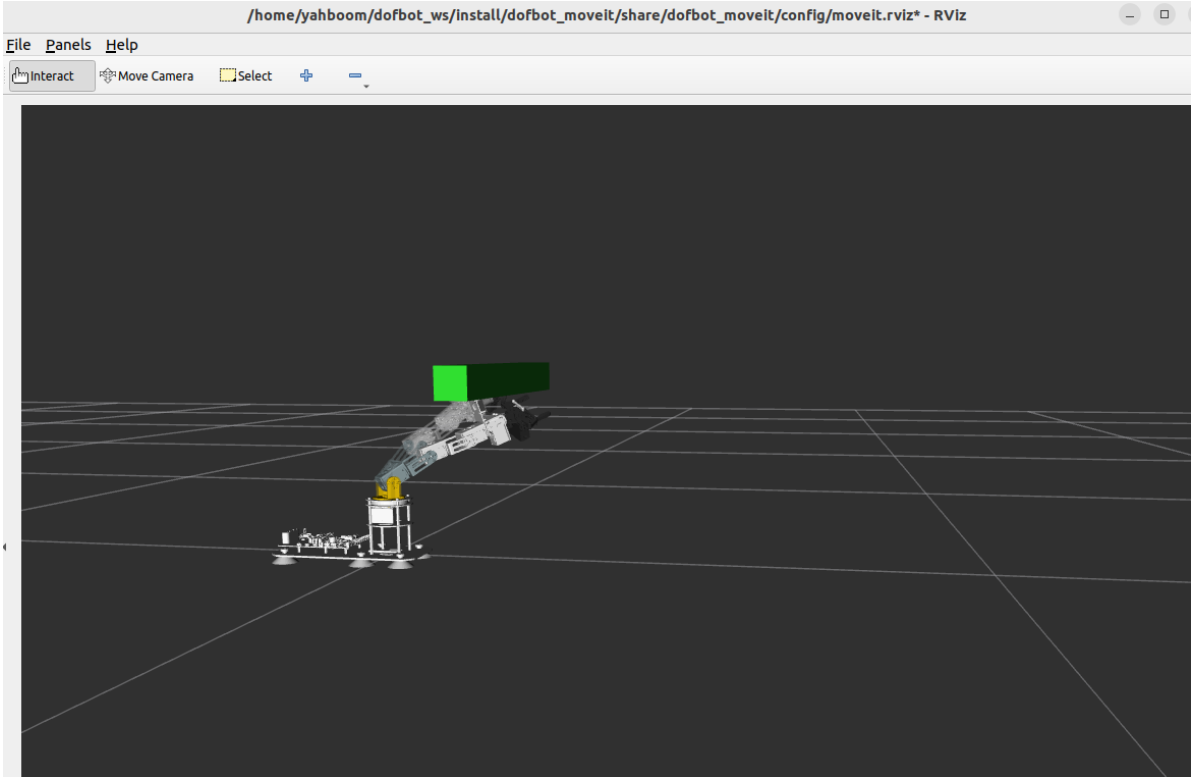
3. Collision Detection

After the program runs, RViz2 will add a cuboid obstacle next to the robotic arm. The robotic arm will plan to the target position, and the entire planning and execution process will automatically avoid the obstacle.

3.1. Launch Command

For this to work, the robotic arm must be successfully loaded within MoveIt, and the message `You can start planning now!` must appear. Run the following command: the robotic arm will automatically plan a path to the specified position while avoiding the rectangular obstacle.

```
# Execute this command within the virtual machine
ros2 run dofbot_moveit obstacle_avoidance
```



3.2. Source Code Analysis

Source Code Path:

```
# Path within the virtual machine
~/dofbot_ws/src/dofbot_moveit/src/obstacle_avoidance.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>

class RandomMove : public rclcpp::Node
{
public:
  RandomMove()
    : Node("random_target_move")
  {
    // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {

    int max_attempts = 5;  // Maximum number of planning attempts
    int attempt_count = 0;  // Current attempt count
    // Initialize move_group_interface_ and planning_scene_interface_ within this
function_
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
```

```
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.


    moveit_msgs::msg::CollisionObject collision_object;
    collision_object.header.frame_id = move_group_interface_-
>getPlanningFrame();
    collision_object.id = "box1";


    shape_msgs::msg::SolidPrimitive primitive;
    primitive.type = primitive.BOX;
    primitive.dimensions.resize(3);
    //Set Obstacle Properties
    primitive.dimensions[primitive.BOX_X] = 0.05;
    primitive.dimensions[primitive.BOX_Y] = 0.05;
    primitive.dimensions[primitive.BOX_Z] = 0.5;
    //Set the position of the obstacles.
    geometry_msgs::msg::Pose box_pose;
    box_pose.orientation.w = 0.7071;
    box_pose.orientation.x = 0.7071;
    box_pose.position.x = 0.20;
    box_pose.position.y = 0.0;
    box_pose.position.z = 0.30;
    //Add Obstacles
    collision_object.primitives.push_back(primitive);
    collision_object.primitive_poses.push_back(box_pose);
    collision_object.operation = collision_object.ADD;

    std::vector<moveit_msgs::msg::CollisionObject> collision_objects;
    collision_objects.push_back(collision_object);

    RCLCPP_INFO(this->get_logger(), "Add an object into the world");
    planning_scene_interface_->addCollisionObjects(collision_objects);

    //RCLCPP_INFO(this->get_logger(), "frame_id =
%s",collision_object.header.frame_id);
    //RCLCPP_INFO(this->get_logger(), "collision_object.id =
%s",collision_object.id);
    // Plan Path
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;

    while (attempt_count < max_attempts)
    {
        attempt_count++;
        // Set Target Joint Space Values
        move_group_interface_->setNamedTarget("up");
            // Plan Route
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
```

```
          // Visualizing Trajectories in RViz
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        attempt_count = 0;
        break;
        }
        else
        {
        RCLCPP_INFO(this->get_logger(), "Planning failed!");
        }
    }


    while (attempt_count < max_attempts)
    {
        attempt_count++;
        // Move the robotic arm to the preset "down" pose.
        move_group_interface_->setNamedTarget("down");
            // Plan Route
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
         bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
          //Visualizing Trajectories in RViz
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        attempt_count = 0;
        break ;
        }
        else
        {
        RCLCPP_INFO(this->get_logger(), "Planning failed!");
        }
    }


    while (attempt_count < max_attempts)
    {
        attempt_count++;
        // Move the robotic arm to the preset "up" pose.
        move_group_interface_->setNamedTarget("up");
            // Plan Route
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
          // Visualizing Trajectories in RViz
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        attempt_count = 0;
        break;
        }
        else
        {
        RCLCPP_INFO(this->get_logger(), "Planning failed!");
        }
```

```
    }

  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<RandomMove>();

  // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```